

CS 614 - Technical Report: Object Detection with Faster R-CNN

Gary Pham - gp492

January 2026

1 Description of the Pretrained Model

1.1 Architecture

For this assignment I used the **Faster R-CNN with ResNet-50 FPN backbone** provided by torchvision (`fasterrcnn_resnet50_fpn` with `FasterRCNN_ResNet50_FPN_Weights.DEFAULT`).

At a high level, the architecture consists of:

- **Backbone (ResNet-50 + FPN)**

A deep residual convolutional network (ResNet-50) extracts feature maps from the input image. A **Feature Pyramid Network (FPN)** takes intermediate feature maps from the backbone and builds a multi-scale feature pyramid, which helps detect both small and large objects.

- **Region Proposal Network (RPN)**

Slides small convolutional filters over the FPN feature maps. At every spatial location it predicts a set of **anchor boxes** (with different aspect ratios and scales), **objectness scores** (object vs. background), and **box regression offsets**. The RPN produces a set of **region proposals** (candidate bounding boxes that may contain objects).

- **RoI Pooling / RoI Align + Detection Head**

Region proposals are projected back onto the FPN feature maps. For each proposal, **RoIAlign** extracts a fixed-size feature representation. These RoI features are passed through fully connected layers that output a **classification distribution** over all object classes plus background, and **bounding-box regression deltas** to refine the proposal boxes.

During inference, the model: (1) runs the backbone and FPN on the input image; (2) uses the RPN to generate proposals; (3) applies the detection head to each proposal; (4) applies **non-maximum suppression (NMS)** per class to remove highly overlapping boxes; and (5) returns final **bounding boxes**, **class labels**, and **confidence scores**.

1.2 Training Dataset

The pretrained weights `FasterRCNN_ResNet50_FPN_Weights.DEFAULT` were trained on the **COCO (Common Objects in Context)** dataset.

- **Type:** Large-scale everyday object detection dataset.
- **Classes:** 91 object categories in the original COCO annotations; 80 categories are commonly used in many implementations (e.g., person, car, bicycle, dog, cat, chair, traffic light).
- **Annotations:** Each image has one or more objects with bounding boxes, category labels, and instance IDs.
- **Image characteristics:** Complex, real-world scenes with multiple objects; large variation in scale, pose, lighting, and occlusion.

Because the model is trained on COCO, it can detect a wide variety of common objects in generic scenes, which is suitable for the test images used in this assignment.

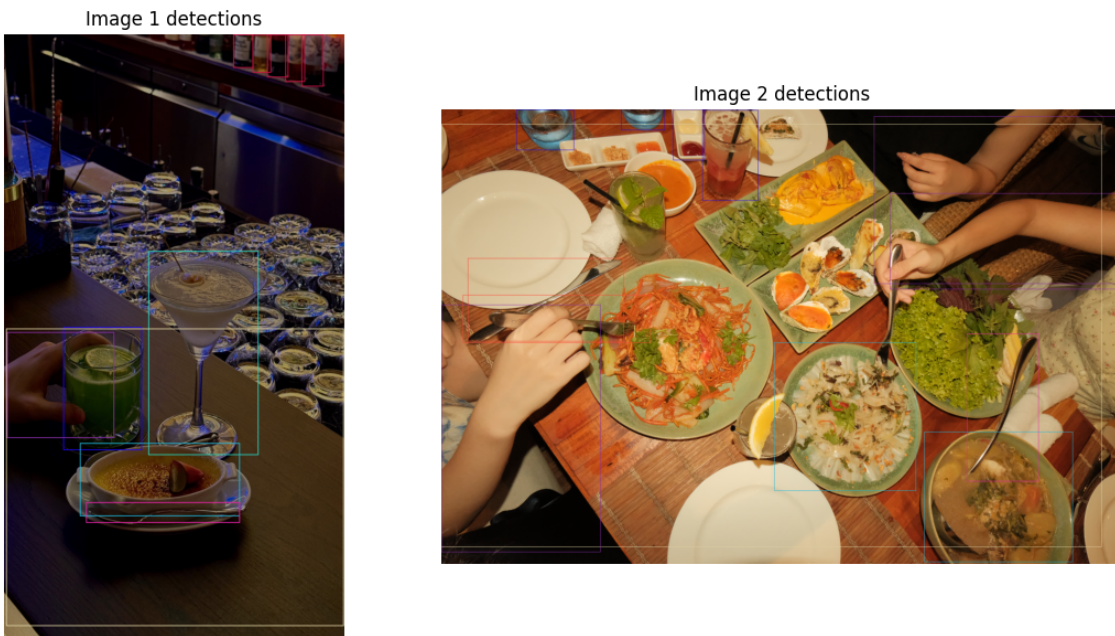
2 2×2 Grid of Images

I selected two test images from my own collection. Both were loaded with PIL, converted to tensors, and preprocessed using the model's default `weights.transforms()` before inference.

Figure 1 shows a 2×2 grid: the first row contains the **original input** images; the second row shows the **detection results** for the same images with color-coded bounding boxes. Detections were filtered by a confidence threshold (see Section 3.2). Bounding boxes were drawn using `draw_bounding_boxes` from `torchvision.utils` with class-specific colors.



(a) Original input images for both Image 1 and Image 2 (combined in `origin.png`).



(b) Combined detection results for both images in a single image (`boxed.png`).

Figure 1: Top row: original input images (from `images/1.jpg`, `images/2.jpg`). Bottom row: detection results with bounding boxes (from `images/1_detections.png`, `images/2_detections.png`); score threshold 0.8.

2.1 Detection Tables (score threshold = 0.8)

Table 1 lists all detections above the threshold for Image 1; Table 2 for Image 2. Coordinates are in pixel space $[x_1, y_1, x_2, y_2]$.

Table 1: Detections for Image 1 with score_threshold=0.8.

Idx	Class	Score	Box $[x_1, y_1, x_2, y_2]$
0	wine glass	0.991	[652, 979, 1148, 1899]
1	cup	0.990	[268, 1321, 622, 1878]
2	bowl	0.985	[343, 1845, 1065, 2176]
3	dining table	0.971	[10, 1329, 1533, 2671]
4	bottle	0.958	[1270, 2, 1359, 214]
5	bottle	0.942	[1034, 0, 1124, 151]
6	bottle	0.920	[1187, 1, 1283, 195]
7	spoon	0.915	[370, 2112, 1064, 2206]
8	person	0.914	[13, 1344, 497, 1823]
9	bottle	0.894	[1343, 1, 1442, 235]
10	bottle	0.886	[1122, 3, 1205, 174]

Table 2: Detections for Image 2 with score_threshold=0.8.

Idx	Class	Score	Box $[x_1, y_1, x_2, y_2]$
0	cup	0.993	[2298, 17, 2790, 813]
1	person	0.965	[3948, 747, 5957, 1589]
2	bowl	0.964	[4250, 2840, 5558, 3983]
3	cup	0.950	[2041, 8, 2317, 457]
4	fork	0.936	[234, 1316, 1606, 2046]
5	dining table	0.931	[0, 140, 5812, 3851]
6	person	0.929	[0, 1725, 1402, 3897]
7	person	0.921	[3807, 70, 5988, 1542]
8	cup	0.916	[1584, 6, 1968, 193]
9	spoon	0.890	[4626, 1979, 5257, 3275]
10	cup	0.882	[662, 12, 1168, 368]
11	bowl	0.871	[2931, 2055, 4177, 3359]
12	fork	0.813	[189, 1638, 1839, 2059]

3 Additional Details, Resources, and Threshold Choice

3.1 Pre-processing and Post-processing

Pre-processing: Images were loaded using PIL. Each image was converted to a tensor with `pil_to_tensor` and passed through `weights.transforms()`, which converts to float, scales to $[0, 1]$, and normalizes using the mean and standard deviation used when training the model.

Post-processing: The model returns, per image, `boxes`, `labels`, and `scores`. I filtered detections by a confidence threshold, mapped `labels` to class names via `weights.meta["categories"]`, and passed the filtered boxes and labels to `draw_bounding_boxes` to produce the final annotated images.

3.2 Threshold Choice and Rationale

I used a **score threshold of 0.8**.

- **Why 0.8?** Lower thresholds (e.g., 0.3–0.5) produced many boxes, including false positives and uncertain detections. A threshold of 0.8 keeps only detections where the model is relatively confident. For my test images, 0.8 preserved the main, visually obvious objects (e.g., people, cars, bottles, tableware) while removing most spurious detections.

- **Effect of different thresholds:**

- **Threshold** < 0.5 : Many more boxes; overlapping and wrong detections; visualization becomes cluttered.
- **Threshold** ≈ 0.7 –**0.8**: Good balance between precision and readability; boxes mostly correspond to clearly visible objects.
- **Threshold** > 0.9 : Very few detections; some real objects with slightly lower confidence are dropped.

Given the goal of visualizing clear, high-quality detections, I chose **0.8** as a reasonable compromise: it preserves the most important objects while avoiding clutter and obvious false positives, making the 2×2 grid easy to interpret.

3.3 Resources Used

- PyTorch tutorial (torchvision detection/finetuning):
https://pytorch.org/tutorials/intermediate/torchvision_tutorial.html
- Torchvision visualization utilities (`draw_bounding_boxes`, etc.):
https://pytorch.org/vision/main/auto_examples/others/plot_visualization_utils.html
- Official PyTorch / Torchvision documentation for `fasterrcnn_resnet50_fpn` and `FasterRCNN_ResNet50_FPN_Weights`, and image transforms.